



| 300
ЛЕТ СПбГУ

Определение игрового опыта по отзывам в Steam

БД315

Сериков Кирилл

Описание задачи

Из Steam собираются тексты отзывов и публичные метаданные, затем по ним предсказывается один из классов опыта: `newbie` (новичок), `casual` (любитель), `engaged` (вовлечённый), `veteran` (ветеран).

Целевая переменная: `experience_class`.

Основные метрики:

- `macro F1` - главная метрика, потому что классы несбалансированы.
- `accuracy` - дополнительная метрика.

Ссылка на GitHub: <https://github.com/romplle/ml-steam-reviews>

Сбор данных

Данные собираются через публичный Steam endpoint.

Настройки сбора данных:

- REVIEWS_PER_GAME = 1000
- LANGUAGE = "english"
- REVIEW_FILTER = "all"
- DAY_RANGE = 365
- REVIEW_TYPE = "all"
- PURCHASE_TYPE = "all"

Используемые поля

Текст: review.

Контекст игры: game, game_group.

Реакции на отзыв: votes_up, votes_funny, weighted_vote_score, comment_count.

Покупка: steam_purchase, received_for_free, written_during_early_access.

Автор и опыт: author_num_games_owned, author_num_reviews, playtime_at_review, playtime_forever.

Группы игр

Игры разделены на две группы, потому что одинаковое количество часов означает разный опыт в одиночных и соревновательных играх.

- Offline: The Witcher 3: Wild Hunt, Red Dead Redemption 2, God of War, The Last of Us Part I, BioShock Infinite, Black Myth: Wukong.
- Competitive: Counter-Strike 2, Dota 2, PUBG: Battlegrounds, Apex Legends, Team Fortress, 2 War Thunder.

Целевая переменная

experience_class строится по времени игры в часах. Для offline и competitive игр заданы разные границы классов.

Класс	offline	competitive
newbie	< 10 ч	< 100 ч
casual	10-50 ч	100-500 ч
engaged	50-100 ч	500-2000 ч
veteran	100+ ч	2000+ ч

Подготовка данных

Очистка текста: переносы строк, лишние пробелы, пустые отзывы.

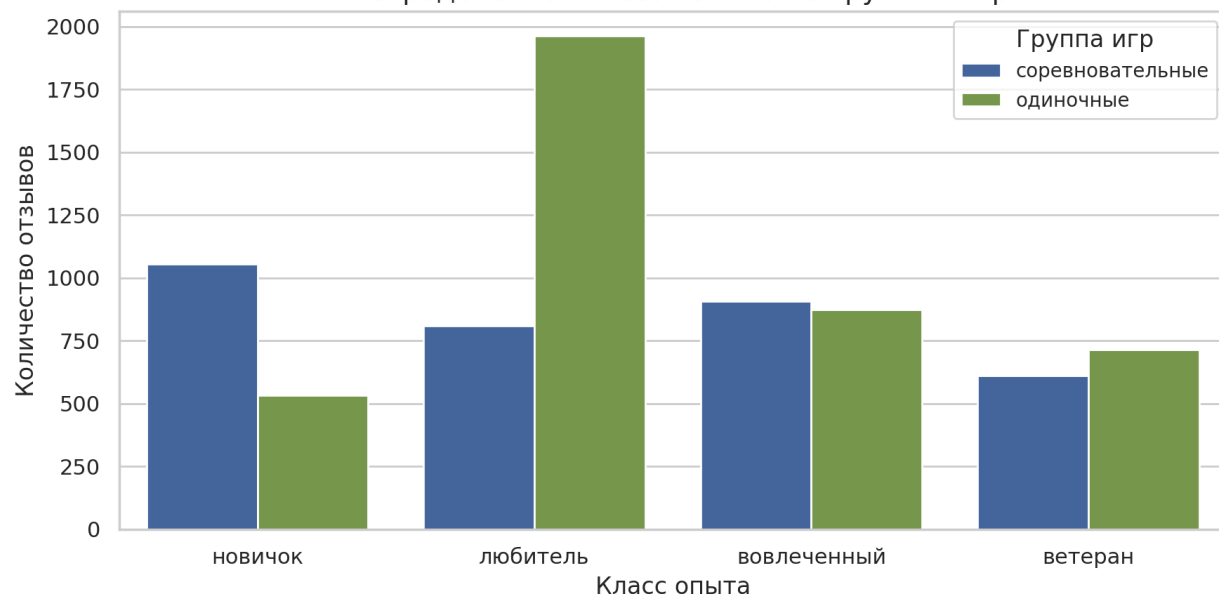
Перевод минут в часы и создание `experience_class`.

Фильтр коротких отзывов: минимум 20 слов и 100 символов.

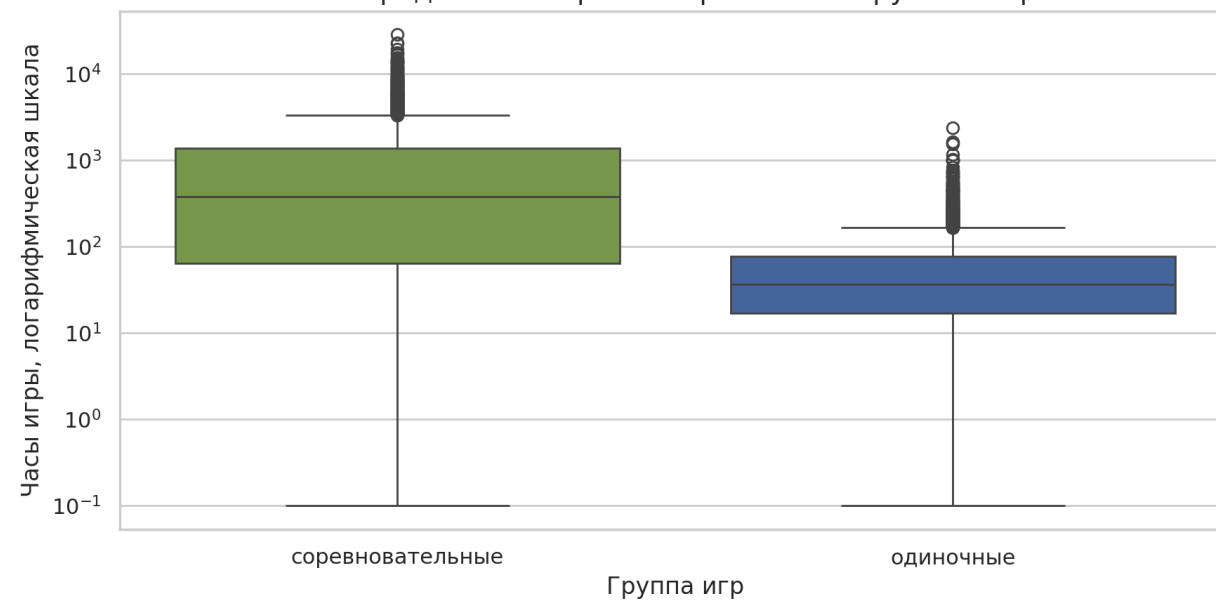
Создание `review_length_chars` и `review_length_words`.

EDA: распределения

Распределение классов опыта по группам игр



Распределение игрового времени по группам игр



Метаданные

Числовые признаки: `SimpleImputer(strategy='median') + StandardScaler(with_mean=False)`.

Категориальные признаки:
`SimpleImputer(strategy='most_frequent') + OneHotEncoder(handle_unknown="ignore")`.

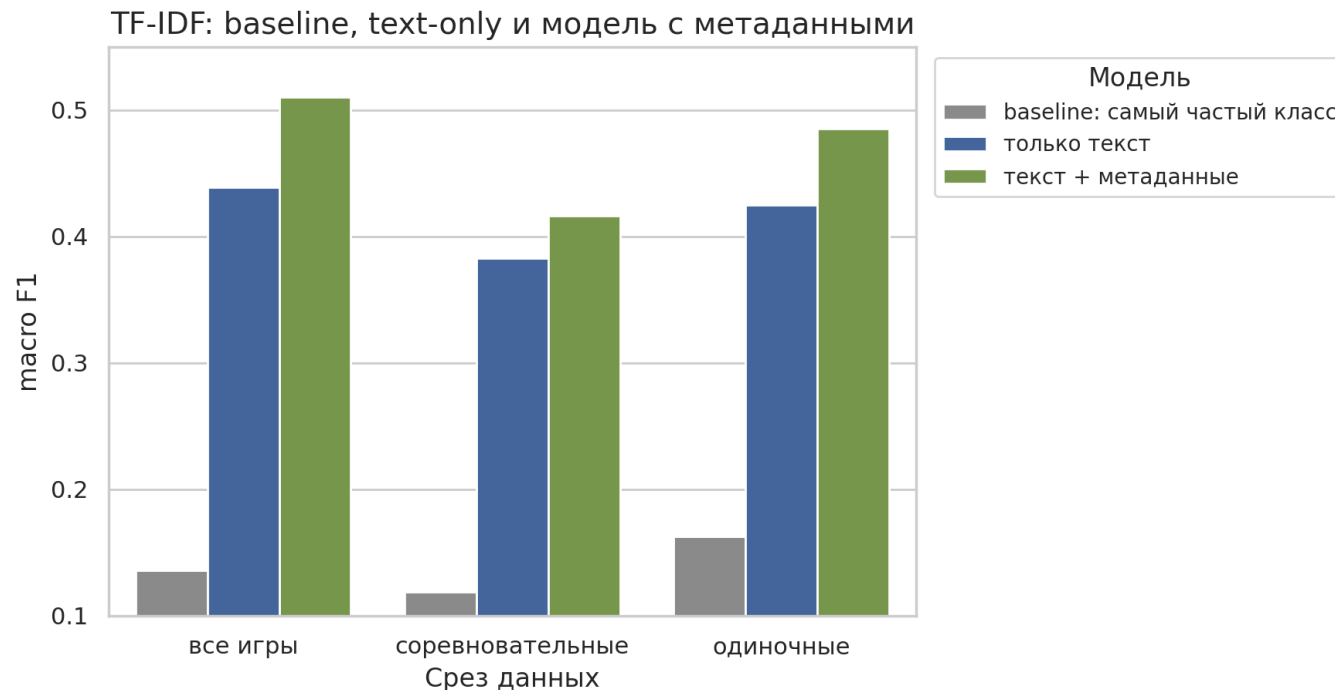
Классификатор для всех данных: `LogisticRegression` с `class_weight='balanced'`.

TF-IDF

Текст представляется через униграммы и биграммы из review.

Параметры: $\text{min_df}=3$,
 $\text{max_df}=0.85$,
 $\text{max_features}=50000$.

Роль метода: сильный и интерпретируемый классический baseline.

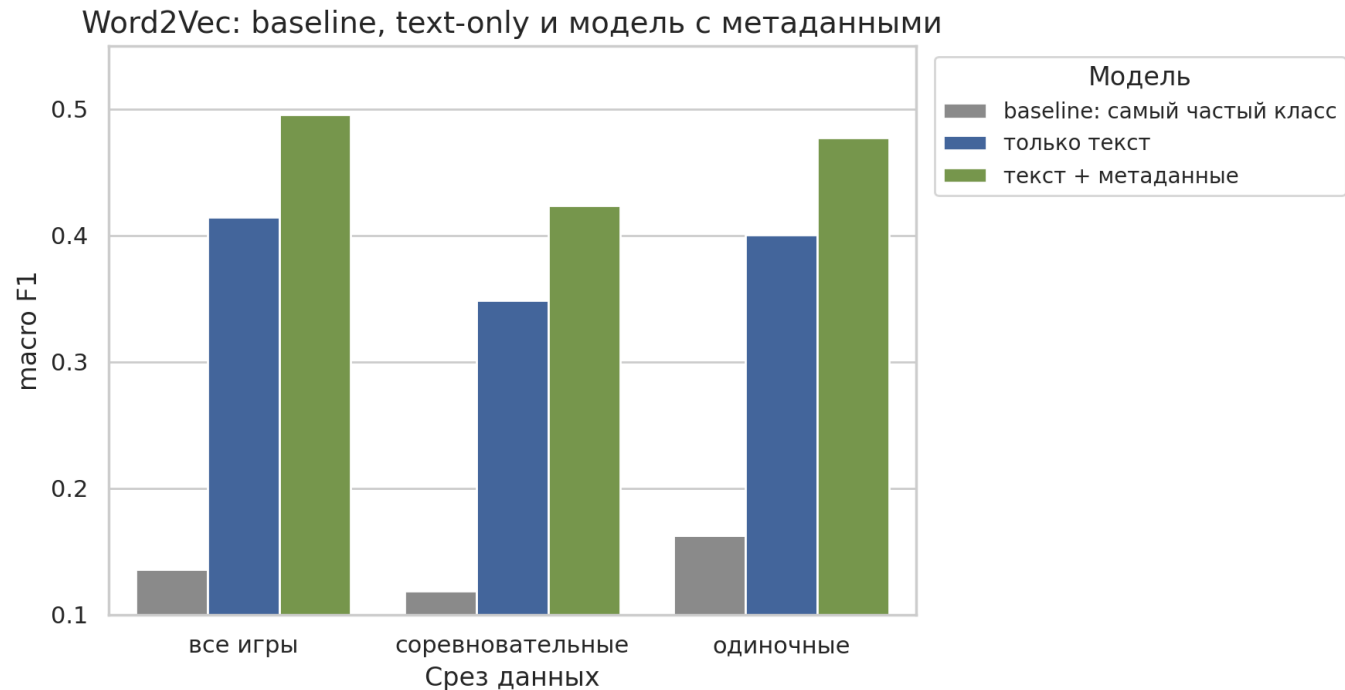


Word2Vec

Текст представляется как среднее Word2Vec-векторов токенов.

Word2Vec обучается только на train-части каждого разбиения.

Роль метода: классический embedding-подход без нейронных моделей.

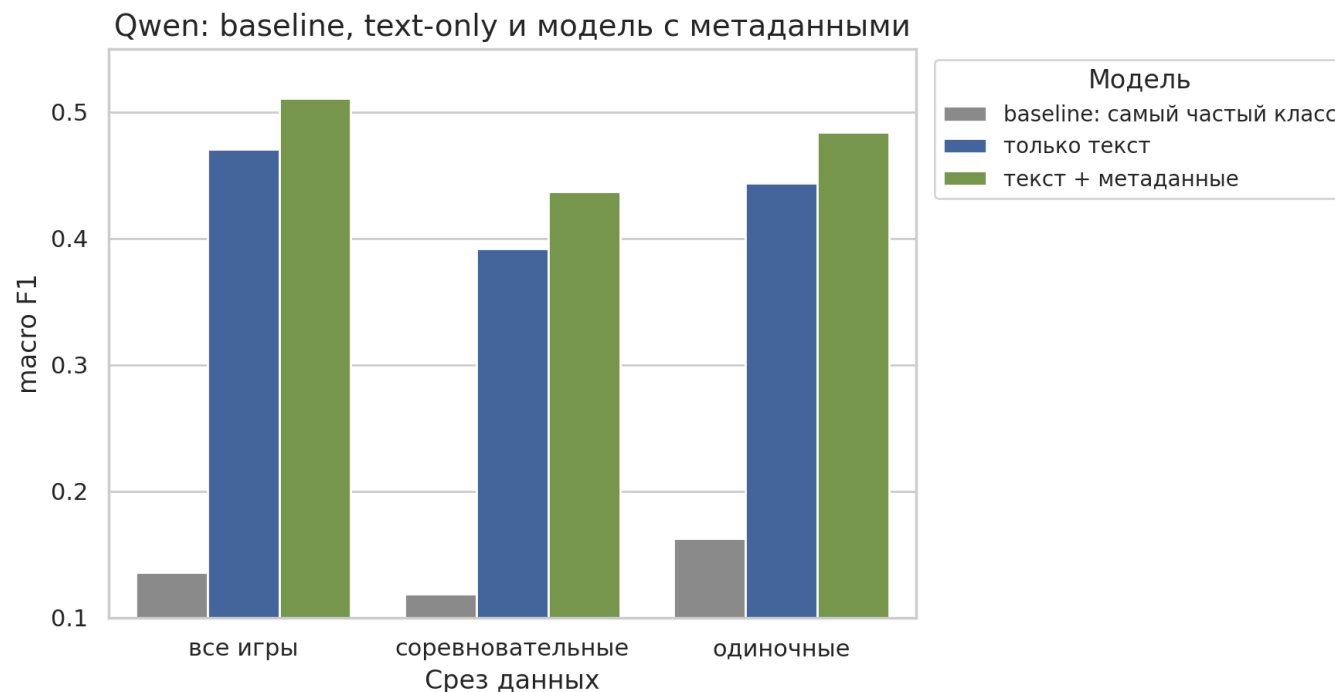


Эмбеddинг-модель Qwen3

Используется Qwen3-Embedding-0.6B.

Qwen только строит эмбеddинги текста; модель не дообучается.

Роль метода: проверить дают ли предобученные семантические представления текста преимущество.



Выводы

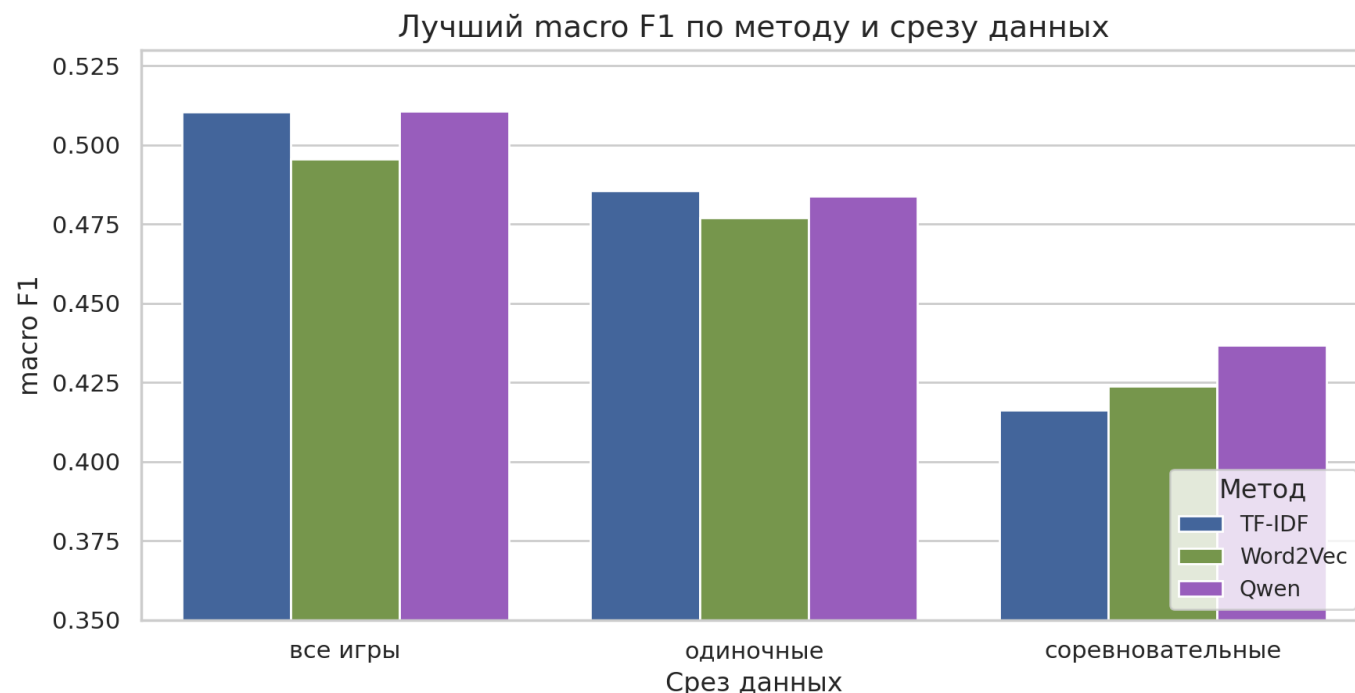
На общем срезе Qwen и TF-IDF практически равны, Qwen совсем немного выше.

Qwen лучше работает на competitive.

TF-IDF лучше работает на offline.

Word2Vec слабее на общем срезе, но полезен как классический сравнительный метод.

Метаданные дают заметный вклад.



**СПАСИБО
ЗА
ВНИМАНИЕ!**